

important to first think about why technical debt happens in the first place.

Ultimately, the decisions made by a manager, architect, or developer introduce technical debt in a software system. For instance, when managers create or modify a project plan, they can decide whether to squeeze in more features in a given time span or to allocate time for tasks such as design reviews and refactoring that can ensure high design quality. Similarly, an architect and a developer have to make numerous technical decisions when designing or implementing the system. These design or code-level decisions may introduce technical debt.

Now, the question is: Why do managers, architects or developers make the decisions that introduce technical debt in the software system? In addition to lack of awareness about technical debt, the software engineering community has identified several common causes that lead to technical debt, such as:

- **Schedule pressure:** Often, while working under deadline pressures to 'get-the-work-done' as soon as possible, programmers resort to hasty changes. For example, they embrace 'copy-paste programming', which helps get the work done. They think that as long as there is nothing wrong syntactically and the solution implements the desired functionality, it is an acceptable approach. However, when such code duplication accumulates, the design becomes incomprehensible and brittle. Thus, a tight schedule for the release of a product with new features can result in a product that has all the desired features but has incurred huge technical debt.
- **Lack of good/skilled designers:** Fred Brooks, in his classic book 'The Mythical Man Month', stressed the importance of good designers for a successful project. If designers lack understanding of the fundamentals of software design and principles, their designs will lack quality. They will also do a poor job while reviewing their team's designs and end up mentoring their teams into following the wrong practices.
- **Not adequately applying design principles:** Developers without the awareness or experience of actually applying sound design principles often end up writing code that is difficult to extend or modify.
- **Lack of awareness about design smells and refactoring:** Many developers are unaware of design smells that may creep into the design over time. These are indicative of poor structural quality and contribute to technical debt. Design smells can be addressed by timely refactoring. However, when developers lack awareness of refactoring and do not perform it, the technical debt accumulates over time. Often, given the different cost and schedule constraints of a project, it may be acceptable to temporarily incur some technical debt. However, it is critical to pay off the debt as early as possible.

How to manage technical debt

It is impossible to avoid technical debt in a software system; however, it is possible to manage it. This section provides a brief overview of high-level steps required to manage technical debt.

Increasing awareness of technical debt: Awareness is the first step toward managing technical debt. This includes awareness of the concept of technical debt, its different forms, the impact of technical debt, and the factors that contribute to it. Awareness of these concepts will help your organisation take well-informed decisions to achieve both project goals and quality goals.

Detecting and repaying technical debt: The next step is to determine the extent of technical debt in the software product. Identifying specific instances of debt and their impact helps prepare a systematic plan to recover from the debt. These two practical aspects of managing technical debt are addressed in detail in Chapter 8 of the book.

Prevent accumulation of technical debt: Once technical debt is under control, all concerned stakeholders must take steps to ensure that the technical debt does not increase and remains manageable in the future. To achieve this, the stakeholders must collectively track and monitor the debt and periodically repay it to keep it under control. 



Note: 'Refactoring for Software Design Smells: Managing Technical Debt', published by Morgan Kaufmann/Elsevier, is available worldwide from November 2014 ([URL: http://amzn.com/0128013974](http://amzn.com/0128013974)). This extract is published with permission from Morgan Kaufmann/Elsevier, MA, USA.

References

- [1] Cunningham W. 'The WyCash portfolio management system'. In: Addendum to the proceedings on object-oriented programming systems, languages, and applications. OOPSLA '92; 1992.
- [2] Highsmith J. 'Zen and the art of software quality'. In: Agile2009 conference; 2009.
- [3] Kelton L. 'Why banks are likely to face more software glitches in 2013'. Available at: <http://www.bbc.co.uk/news/technology-21280943>.
- [4] Brooks Jr F P. 'The Mythical Man-month'. Anniversary ed. Boston, MA, USA: Addison-Wesley Longman Publishing Co Inc; 1995./

By: Girish Suryanarayana Ganesh Samarthyam and Tushar Sharma

Girish Suryanarayana is a senior research scientist at Research and Technology Centre, Siemens Technologies and Services Pvt Ltd, Bengaluru.

Ganesh Samarthyam is an independent consultant and corporate trainer based in Bengaluru. He has contributed numerous articles to LFY/OSFY since 2006.

Tushar Sharma is a technical expert at Research and Technology Centre, Siemens Technologies and Services Pvt Ltd, Bengaluru.